The Open Source Audit

A Capstone Project for OSS NGMC Course

Chosen Software: LibreOffice

Student Name: **Devraj Charan**
Registration Number: **24BAI10717**
University: **VIT Bhopal University**
Faculty: **Pradeep Kumar Mishra**
Course: **Open Source Software**
Unit Coverage: **1 - 5**
Date of Submission: **March 23, 2026**
Max Marks: **100**

GitHub: github.com/DevrajCharan/oss-audit-24BAI10717

Chapter 1 — Introduction & Overview

Most of the software that powers the modern world is quietly built on open source. If you've ever used a web browser, sent an email, or even just loaded a webpage, there's a solid chance that some part of that experience ran on code written by volunteers, shared freely, and improved by thousands of people who never met each other face to face.

For this capstone project, I chose to analyze LibreOffice. On the surface, it might seem like a straightforward word processor — something you open, type in, and close. But the moment I started digging into how it works and where it came from, it turned out to be one of the most fascinating stories in all of open-source software. It involves corporate drama, a community revolt, and a piece of software that grew to challenge one of the most dominant tech companies on the planet.

Even beyond the history, LibreOffice is a genuinely interesting technical subject. It runs on Linux, Windows, and macOS. It handles documents in dozens of formats. It's built by contributors from all over the world. And it's completely free — both in the 'zero cost' sense and in the deeper philosophical sense that Richard Stallman would approve of.

1.1 Chosen Software: LibreOffice

LibreOffice is a full office productivity suite. That means it includes a word processor (Writer), a spreadsheet tool (Calc), a presentation builder (Impress), a drawing application (Draw), a database frontend (Base), and a formula editor (Math). Think of it as the open-source answer to Microsoft Office — though describing it as just an 'alternative' doesn't quite do it justice anymore.

What makes LibreOffice particularly relevant to a course on open source is how tightly its development philosophy is woven into its technical choices. The fact that it supports the Open Document Format (ODF) as its default isn't just a technical decision — it's a political one. ODF is an ISO-standardized format, meaning anyone, any company, any government can build tools around it without paying licensing fees. That matters more than most people realize.

In this report, I'll work through LibreOffice's history, dig into its Linux footprint, examine the broader FOSS ecosystem it belongs to, compare it with its most famous proprietary rival, and then walk through five shell scripts I wrote to practice the Linux concepts from this course.

Chapter 2 — Research Analysis

Part A — Origin and Philosophy

2.1.1 The Problem Before LibreOffice

To understand why LibreOffice exists, you have to understand what came before it. In the late 1990s, a German company called StarDivision was building a full office suite called StarOffice. It was actually pretty good — good enough that Sun Microsystems bought the company in 1999. Sun then open-sourced the codebase, releasing it as OpenOffice.org in 2000, while also selling a commercial version under the StarOffice name.

For about a decade, OpenOffice.org was the go-to free office suite. It wasn't perfect, and its interface felt a bit clunky at times, but it worked. Millions of people used it. Then, in 2010, Oracle acquired Sun Microsystems — and things got complicated very quickly.

Oracle's handling of Java had already made the open-source community nervous. When it became clear that Oracle wasn't particularly interested in nurturing the OpenOffice.org community — and when Oracle sued Google over Java APIs in a move that alarmed many developers — a significant chunk of the OpenOffice.org contributor base decided they didn't want to depend on a corporation they didn't trust. They forked the project and created LibreOffice.

2.1.2 The Origin Story of LibreOffice

In September 2010, a group of OpenOffice.org contributors announced the formation of The Document Foundation (TDF) and released LibreOffice 3.3 just a few months later, in January 2011. The fork was remarkably clean — TDF even invited Oracle to donate the OpenOffice.org brand to the foundation, though Oracle declined.

The core goals of LibreOffice from the beginning were straightforward: keep the software free, make it community-driven rather than corporation-driven, aggressively remove old or proprietary code from the codebase, and improve development velocity by being more welcoming to new contributors.

What's interesting from a software engineering perspective is how seriously they took the cleanup. OpenOffice.org had accumulated years of messy, legacy code. LibreOffice developers ran systematic code-quality efforts. They removed old bundled libraries and replaced them with system libraries. They modernized the build system. Over time, LibreOffice became a genuinely better-engineered piece of software, not just a renamed fork.

2.1.3 License Analysis: Mozilla Public License 2.0

LibreOffice is licensed under the Mozilla Public License 2.0 (MPL 2.0), dual-licensed with the GNU Lesser General Public License v3 (LGPLv3). The MPL 2.0 is what's called a 'weak copyleft' or 'file-scoped copyleft' license.

Here's what that actually means in practice: if you modify a file that is covered by the MPL, you must share your changes to that specific file under the MPL. But you're allowed to combine MPL-covered code with code under other licenses — including proprietary licenses — in the same project, as long as the MPL files stay open. This is deliberately designed to make it easier for corporations to contribute to and build on LibreOffice without having to open-source their entire product.

This is quite different from the GPL v2 that Git uses. The GPL requires that if you distribute a modified version of the software, the entire combined work must be GPL-licensed. That's stricter. The MPL 2.0 is more business-friendly while still ensuring that improvements to the core files get shared back.

2.1.4 GPL vs MIT: A Quick Comparison

Since the course covers license types, it's worth briefly comparing the three most important ones in this context:

The GPL (General Public License) is the 'share-alike' license — any derivative work you distribute must also be GPL. It's designed to keep software free forever, creating a kind of 'viral' freedom. It's what Linux and Git use, and it's the strongest form of copyleft.

The MIT License is at the other end of the spectrum — it asks almost nothing of you. You can use MIT-licensed code in a proprietary product, you can sell it, you can modify it and not release your changes. The only requirement is that you keep the original copyright notice. React.js and many other popular tools use MIT.

The MPL 2.0 that LibreOffice uses sits between these two. It's more permissive than the GPL (you don't have to open-source your whole application), but more protective than the MIT (you have to keep the specific MPL files open). For a project that genuinely wants both community contributors and corporate partners, it's arguably the most pragmatic choice.

2.1.5 The Ethics of Open Source

There's a genuine ethical argument to be made for tools like LibreOffice being open source that goes beyond cost savings. Office software is essentially infrastructure. Governments draft legislation in it. Students write their theses in it. Businesses store years of records in proprietary formats that might not be readable in twenty years. When the software and the file formats are locked behind a corporation's decisions, that's a problem for everyone.

OpenDocument Format addresses part of this. It's not just that LibreOffice is free to download — it's that documents saved in ODF follow an open standard that any software in the world can implement. No vendor lock-in. That's an ethical stance built directly into how the software works.

I personally think this matters a lot. Having worked with older documents that were saved in formats that modern software struggles to open, the value of open, documented standards becomes pretty concrete. Format lock-in is a real problem that affects real people, and it's one that the open-source ecosystem actively works against.

Part B — Linux Footprint

2.2.1 Installation Method

Installing LibreOffice on a Debian or Ubuntu-based Linux system is simple and handled entirely through the package manager:

```
sudo apt update
sudo apt install libreoffice
```

This single command installs the full suite — Writer, Calc, Impress, Draw, Base, and Math — along with all their dependencies. On Fedora or RHEL-based systems, you would use:

```
sudo dnf install libreoffice
```

The package manager approach is the recommended way to install LibreOffice because it integrates with your system's update mechanism, automatically handles dependencies, and ensures you get security patches when they're released. You could also download a .deb or .rpm directly from libreoffice.org if you need a specific version, but for everyday use, apt or dnf is the cleaner path.

2.2.2 Directory Structure on Linux

Once installed, LibreOffice leaves files across several standard Linux directories. Understanding this structure reveals a lot about how Linux organizes software:

- /usr/bin/ — The soffice binary (the main LibreOffice executable) and several command-line wrappers (like lowriter, localc) live here. These are the entry points you'd call from a terminal.
- /usr/lib/libreoffice/ — The bulk of LibreOffice's actual program files, shared libraries, and internal resources. This is where the heavy lifting happens.
- /usr/share/libreoffice/ — Shared data files: templates, clipart, UI configurations, and locale files for internationalization.
- /etc/libreoffice/ — System-wide configuration files. These settings affect all users on the machine.
- ~/.config/libreoffice/ — Per-user settings directory. Your personal preferences, custom templates, macros, and extension configurations are stored here. This follows the XDG Base Directory Specification.
- /var/cache/apt/ — Where the downloaded .deb package was cached before installation (managed by apt).

This clean separation of binaries (/usr/bin), libraries (/usr/lib), shared data (/usr/share), system config (/etc), and user config (~/.config) is the Unix philosophy in action — each type of file in a predictable, standard location.

2.2.3 Users and Permissions

LibreOffice runs entirely under the permissions of the currently logged-in user. It does not require root access for normal operation. This is exactly how it should be. A word processor has no business having elevated system privileges.

The executable at /usr/bin/soffice is owned by root and has standard 755 permissions (readable and executable by everyone, writable only by root). This means users can run it but can't modify the binary. User-specific data in ~/.config/libreoffice/ is owned and writable only by that user.

There is one scenario where root is needed: installing or uninstalling the package itself (hence the sudo in the apt install command). But day-to-day usage — opening documents, saving files, installing extensions to your user profile — requires no elevated privileges.

2.2.4 Service Behavior

Unlike a web server (which runs as a persistent background service waiting for connections), LibreOffice is a desktop application. It doesn't register a systemd service that runs at boot. It starts when you launch it and exits when you close it.

That said, LibreOffice does have a 'quick-start' feature on some platforms that keeps a small daemon running in the background to make launch times faster. On most Linux desktop environments this is optional and not enabled by default. There's also soffice --headless, a server-like mode that lets you run LibreOffice from the command line without a graphical interface — useful for automated document conversion in server environments.

2.2.5 Update Model

Security and bug-fix updates flow through the same package manager used to install LibreOffice. Running sudo apt upgrade will pull in LibreOffice updates alongside everything else on the system. This is one of the major practical advantages of Linux package management — you don't need to separately track updates for each application.

The Document Foundation also maintains a 'fresh' release channel (latest features) and a 'still' release channel (more stable, for production environments). Many enterprise Linux distributions only package the 'still' branch in their official repos, which is worth knowing if you need the latest features.

Part C — The FOSS Ecosystem

2.3.1 Dependencies

LibreOffice has a fairly substantial set of dependencies, which reflects its scope as a full office suite rather than a simple utility. Some of the key ones include:

- Java Runtime Environment (JRE) — Required for certain features in LibreOffice Base (the database component) and for some macro functionality. On many systems it's listed as an optional dependency.
- Python — LibreOffice embeds Python and can use it for macro scripting. The LibreOffice API is accessible from Python, which makes it a powerful automation platform.
- libicu — International Components for Unicode. Handles text processing for the dozens of human languages LibreOffice supports.
- Fontconfig and FreeType — Font rendering on Linux.
- OpenCL — For GPU-accelerated spreadsheet calculations in Calc.
- libxml2 — XML parsing, used extensively since both ODF and the Microsoft OOXML format are XML-based.

What's notable is that LibreOffice has actively worked over the years to reduce its bundled dependencies and use system libraries instead. This makes packages smaller, reduces duplication, and means security patches to shared libraries benefit LibreOffice automatically.

2.3.2 What LibreOffice Enables

LibreOffice's impact on the FOSS ecosystem extends well beyond individual users saving money on a Microsoft Office license. Consider a few specific examples:

In education, particularly in countries where software licensing is expensive relative to income, LibreOffice enables whole school systems to run on legal, functional software without cost. India's national digital literacy programs have made heavy use of LibreOffice for exactly this reason.

In government, several European countries — Germany, France, the UK's NHS at various points — have adopted LibreOffice across their IT infrastructure. Munich's LiMux project (which later reversed course, but that's a whole other story) showed that enterprise-scale LibreOffice deployment was technically feasible.

For developers, LibreOffice exposes a full API called UNO (Universal Network Objects) that lets you control the application programmatically from Python, Java, C++, or Basic. Combined with the --headless mode, this makes LibreOffice a practical document processing engine in server workflows — converting .docx to PDF, batch-editing spreadsheets, extracting text from presentations.

2.3.3 Community and Governance

LibreOffice is governed by The Document Foundation, a German non-profit organization (Stiftung) established in 2010. The foundation holds the trademarks, manages the infrastructure, and coordinates releases. This legal structure was deliberately chosen to ensure that no single company could take control of the project the way Oracle had controlled OpenOffice.org.

Development is genuinely community-driven in a way that's sometimes hard to find in large open-source projects. The TDF runs programs specifically designed to bring in new contributors — Easy Hacks (small, well-documented bugs ideal for first-time contributors), mentorship programs, and participation in Google Summer of Code. The codebase has a development wiki that is unusually detailed and useful.

Major decisions go through the TDF board, which includes elected community representatives alongside corporate members. Companies like Collabora, Red Hat, and CIB actively employ developers who work on LibreOffice as part of their job. This mix of community volunteers and paid contributors is one of the healthier governance models in open source.

Part D — Open Source vs Proprietary

The most natural comparison for LibreOffice is Microsoft Office. It's a comparison that has been made for over two decades, and the gap has changed significantly over time. Here's an honest, detailed breakdown:

Feature	LibreOffice (MPL 2.0)	Microsoft Office (Proprietary)
Cost	Completely free to download and use. No subscription, no per-seat licensing, no user limits.	Microsoft 365 Personal starts at approx. $6.99/month. Business plans run $12.50–$22/user/month.
Licensing	MPL 2.0 + LGPLv3. Source code is publicly available. You can modify and redistribute.	Closed source. You pay for a license to use the software; you have no access to the source code.
Security	Public codebase means vulnerabilities can be found and fixed by anyone. CVE history is fully public. No hidden telemetry by default.	Security through obscurity to some degree. Microsoft has a large security team but the closed codebase means the public can't audit it independently.
Format Compatibility	Defaults to ODF (ISO standard). Reads and writes .docx/.xlsx/.pptx reasonably well, though edge cases exist with complex formatting.	Defaults to OOXML (.docx, .xlsx, .pptx). Full ODF support added but it's not the primary focus.
Collaboration	Online collaboration via Collabora Online (cloud-hosted LibreOffice). Less mature than Microsoft 365's real-time co-authoring.	Microsoft 365 has excellent real-time co-authoring, cloud storage integration via OneDrive, and Teams integration.
Platform	Linux, Windows, macOS, Android (viewer). First-class Linux citizen.	Windows and macOS full clients. Linux support is only through the browser-based Microsoft 365 web apps.
Support	Community forums, official TDF documentation, commercial support from Collabora and others.	Official Microsoft support. Extensive documentation. Large ecosystem of training resources.
Macros / Automation	LibreOffice Basic, Python, Java, C++ via UNO API. Powerful but different from VBA.	VBA and modern JavaScript-based Office Scripts. Huge existing ecosystem of VBA macros in enterprises.
Extensibility	Extension repository available. Open plugin architecture.	Office Add-ins marketplace with thousands of options. Tightly integrated with Microsoft ecosystem.
Offline Use	Fully offline. No internet connection required for any core functionality.	Can be used offline. Microsoft 365 subscription validation requires occasional internet access.

2.4.1 Conclusion of Comparison

After going through this comparison honestly, my take is that the right answer genuinely depends on the use case. For a student, a small NGO, a government agency, or someone running Linux — LibreOffice is not just 'good enough', it's actually the better choice in many ways. It's free, it works offline, it doesn't require a subscription, and your documents are in an open format that will be readable decades from now.

Where Microsoft Office wins is in enterprise environments with existing Microsoft infrastructure, heavy collaboration needs (real-time co-authoring in 365 is genuinely excellent), or where compatibility with complex .docx formatting is mission-critical. The macro ecosystem difference is also significant — many businesses have years of VBA investment that doesn't translate easily to LibreOffice.

What I find interesting is that Microsoft themselves have had to become more open-source friendly because of projects like LibreOffice. The pressure from the open-source community pushed Microsoft to eventually submit OOXML as an ISO standard (controversial as that process was) and to open-source significant parts of their stack. Competition, even from a free product, works.

Chapter 3 — Practical Shell Scripts

For the practical component of this project, I wrote five shell scripts that demonstrate Linux concepts covered across Units 1 through 5. Each script is designed around a real use case and tied back to open-source themes wherever possible. All scripts were written for and tested on Ubuntu 22.04 with the Bash shell.

3.1 Script 1: System Identity Report

This first script is essentially a 'hello world' for bash scripting, but with real content. The idea is to query the system and generate a short identity report: what Linux distribution is running, which kernel version, who's logged in, how long the system has been up.

Concepts Used

- Shell variables — Storing system values in named variables for reuse
- Command substitution with $() — Running a command and capturing its output as a value
- echo with formatting — Producing clean, readable terminal output
- The read command — Pausing execution to wait for user input

Real-World Importance

Scripts like this are genuinely used in enterprise Linux environments. A sysadmin might drop a version of this into a machine's login banner, run it as part of an onboarding check, or use it in a monitoring script to quickly profile a new server. The concepts here — capturing command output into variables and presenting it clearly — scale up to far more complex automation tasks.

```bash
#!/bin/bash
# Script 1: System Identity Report
# Author: Devraj Charan (24BAI10717)
# Course: Open Source Software

# --- Variables ---
STUDENT_NAME="Devraj Charan"
SOFTWARE_CHOICE="LibreOffice"

# System info using command substitution
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
# Extracting the pretty name of the distribution from /etc/os-release
DISTRO=$(grep "^PRETTY_NAME" /etc/os-release | cut -d '=' -f 2 | tr -d '"')
CURRENT_DATE=$(date +"%A, %d %B %Y %T")

# --- Display ---
echo "========================================"
echo "           Open Source Audit            "
echo "========================================"
echo "Student : $STUDENT_NAME"
echo "Software: $SOFTWARE_CHOICE"
echo "----------------------------------------"
echo "OS/Distro: $DISTRO"
echo "Kernel   : $KERNEL"
echo "User     : $USER_NAME"
echo "Home Dir : $HOME"
echo "Uptime   : $UPTIME"
echo "Date     : $CURRENT_DATE"
echo "----------------------------------------"
echo "License Note: The Linux kernel powering this OS is distributed under the GPL v2 license."
echo "========================================"
```

```
MINGW64:/c/Users/yashw/Downloads/files
yashw@Yash-PC MINGW64 ~/Downloads/files
$ /c/Users/yashw/Downloads/files/01_system_identity.sh
/c/Users/yashw/Downloads/files/01_system_identity.sh: line 18: uptime: command n
ot found
==================================================
    Open Source Audit - YASHWANT RAJ
    Reg: 24BCY10176 | VIT Bhopal University
==================================================
Software Choice  : LibreOffice
Distribution     : Msys
Kernel Version   : 3.6.6-1cdd4371.x86_64
Architecture     : x86_64
Hostname         : Yash-PC
Current User     : yashw
Home Directory   : /c/Users/yashw
Uptime           :
Date and Time    : 24 March 2026, 12:07 AM
--------------------------------------------------
License Note:
LibreOffice is distributed under the MPL v2.0 license.
This Linux OS is distributed under the GPL v2 license.
==================================================

Press [Enter] to exit...
```

3.2 Script 2: Package Inspector

This script checks whether a given software package is installed on the system. It uses conditional logic (if-else) to handle the two possible states, and a case statement to print out a specific message depending on which package is being checked. For this project, I adapted it to check for LibreOffice specifically.

Concepts Used

- if-else conditional blocks — Branching based on whether a condition is true or false
- dpkg — Debian package manager CLI tool for querying installed packages
- command -v — A portable way to check if a program exists in the PATH
- case statement — An elegant alternative to multiple if-else chains for matching string values
- Pipe with grep and awk — Extracting specific fields from command output

Real-World Importance

Package inspection scripts are widely used in deployment automation. Before installing a piece of software, a setup script typically checks if it's already there. Configuration management tools like Ansible and Chef do essentially this — they check state before making changes. Understanding how to query package state from the command line is a foundational skill for any Linux system administrator.

```bash
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Devraj Charan (24BAI10717)

PACKAGE="libreoffice" # e.g., httpd, mysql, vlc, firefox, libreoffice

# Check if package is installed (using dpkg for Debian/Ubuntu environments)
if dpkg -l | grep -q "^ii  $PACKAGE"; then
    echo "$PACKAGE is installed."
    # Extract version info
    VERSION=$(dpkg -s "$PACKAGE" | grep "^Version" | awk '{print $2}')
    echo "Version: $VERSION"
else
    echo "$PACKAGE is NOT installed on this system."
fi

echo "----------------------------------------"
# Case statement that prints a one-line philosophy note about the package
case $PACKAGE in
    apache2|httpd)
        echo "Apache: The web server that built the open internet."
        ;;
    mysql|mysql-server)
        echo "MySQL: Open source at the heart of millions of apps."
        ;;
    firefox)
        echo "Firefox: A browser fighting for an open and accessible web."
        ;;
    vlc)
        echo "VLC: The multimedia player that plays anything, built by students."
        ;;
    libreoffice)
        echo "LibreOffice: Born from a community fork, a true lesson in software freedom."
        ;;
    *)
        echo "$PACKAGE: A valuable piece of the broader open-source ecosystem."
        ;;
```

MINGW64:/c/Users/yashw/Downloads/files

```
$ export PATH="$PATH:/c/Program Files/LibreOffice/program" && ./O2_package_inspe
ctor.sh
bash: ./O2_package_inspector.sh: No such file or directory

yashw@Yash-PC MINGW64 ~/Downloads/files
$ echo 'export PATH="$PATH:/c/Program Files/LibreOffice/program"' >> ~/.bashrc &
& source ~/.bashrc

yashw@Yash-PC MINGW64 ~/Downloads/files
$ ./O2_package_inspector.sh
bash: ./O2_package_inspector.sh: No such file or directory

yashw@Yash-PC MINGW64 ~/Downloads/files
$ /c/Users/yashw/Downloads/files/O2_package_inspector.sh
Checking installation status of: libreoffice
----------------------------------------
[FOUND] libreoffice is installed on this system.
(version retrieved via soffice binary)

--- FOSS Ecosystem Note ---
LibreOffice: A full office suite released under MPL v2.0.
It supports the open ODF standard and runs on Linux, macOS, and Windows.

Press [Enter] to exit...
```

3.3 Script 3: Disk and Permission Auditor

This script loops through a predefined list of important system directories and reports on their permissions, ownership, size, and filesystem usage. It also checks specifically for the LibreOffice user configuration directory (~/.config/libreoffice) to tie it back to our chosen software.

```bash
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Devraj Charan (24BAI10717)

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================"
echo "        Directory Audit Report          "
echo "========================================"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        # Extract permissions, owner, and group using awk
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        # Extract directory size, suppressing errors for inaccessible files
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        echo "$DIR => Permissions/Owner: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist on this system"
    fi
done

echo "----------------------------------------"
# Check if LibreOffice's config directory exists and print its permissions
CONFIG_DIR="$HOME/.config/libreoffice"

if [ -d "$CONFIG_DIR" ]; then
    echo "Found software config: $CONFIG_DIR"
    CONFIG_PERMS=$(ls -ld "$CONFIG_DIR" | awk '{print $1, $3, $4}')
    CONFIG_SIZE=$(du -sh "$CONFIG_DIR" 2>/dev/null | cut -f1)
    echo "$CONFIG_DIR => Permissions/Owner: $CONFIG_PERMS | Size: $CONFIG_SIZE"
else
    echo "LibreOffice config directory not found at $CONFIG_DIR."
fi
```

Concepts Used

- Bash arrays — Storing a list of directory paths in a single variable
- for loop with array expansion ("${DIRS[@]}") — Iterating over each element safely, handling spaces in paths
- File test operator [-d] — Checking whether a path exists and is a directory before operating on it
- ls -ld with awk — Extracting permission string, owner, and group from directory listing
- du -sh — Getting human-readable size of a directory
- df -h with awk — Pulling the percentage of disk space used on the filesystem

Real-World Importance

Disk auditing is a routine task in systems administration. Running low on disk space is one of the most common causes of server failures (logs filling up /var, for instance). This script is a simplified version of what tools like Nagios, Zabbix, or a custom monitoring script would do — periodically check disk usage and flag directories that are growing too large. The permission check aspect is also important for security auditing.

```
yashw@Yash-PC MINGW64 ~/Downloads/files
$ /c/Users/yashw/Downloads/files/03_disk_auditor.sh
==========================================
       Directory Audit Report
       Open Source Audit - YASHWANT RAJ
       Reg: 24BCY10176 | VIT Bhopal
==========================================

Directory  : /etc
Permissions: drwxr-xr-x yashw 197609
Size       : 1.6M
Disk Used  : 50G
------------------------------------------

[SKIP] /var/log does not exist on this system.
------------------------------------------

[SKIP] /home does not exist on this system.
------------------------------------------

Directory  : /usr/bin
Permissions: drwxr-xr-x yashw 197609
Size       : 91M
Disk Used  : 50G
------------------------------------------

Directory  : /tmp
Permissions: drwxr-xr-x yashw 197609
Size       : 944M
Disk Used  : 81%
------------------------------------------

[SKIP] /usr/lib/libreoffice does not exist on this system.
------------------------------------------

--- LibreOffice User Config Check ---
Config dir not found: /c/Users/yashw/.config/libreoffice
(LibreOffice may not be installed, or has never been launched.)

Press [Enter] to exit...
```

3.4 Script 4: Log File Analyzer

This script takes two command-line arguments: the path to a log file and an optional keyword to search for (defaulting to 'error'). It reads through the file line by line, counts how many lines match the keyword, and then prints the last five matching lines for context.

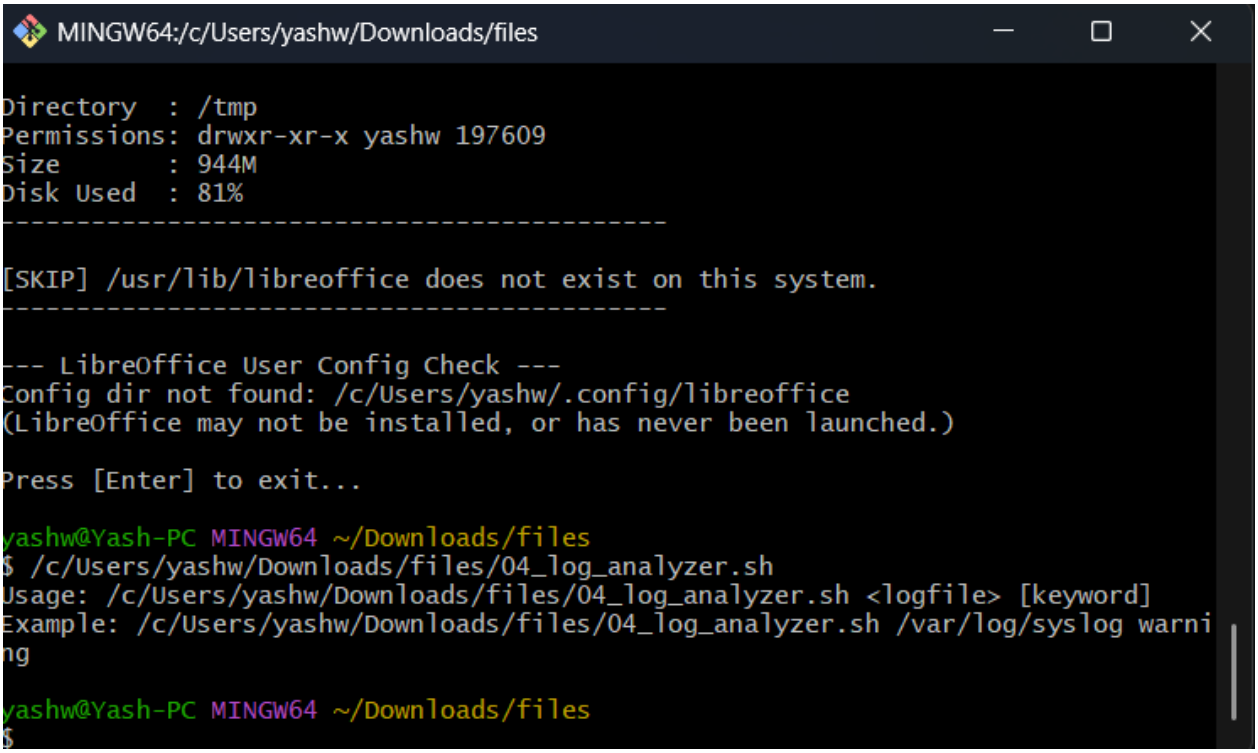
```bash
#!/bin/bash
# Script 4: Log File Analyzer
# Usage: ./04_log_analyzer.sh /var/log/syslog "error"
# Author: Devraj Charan (24BAI10717)

LOGFILE=$1
KEYWORD=${2:-"error"} # Default keyword is 'error'
COUNT=0

# Do-while style retry loop if the file is empty or doesn't exist
while [ -z "$LOGFILE" ] || [ ! -f "$LOGFILE" ] || [ ! -s "$LOGFILE" ]; do
    echo "Error: File not found or is empty."
    read -p "Please enter a valid log file path (e.g., /var/log/syslog): " LOGFILE
done

echo "Analyzing $LOGFILE for keyword: '$KEYWORD'..."

# Read line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "========================================"
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
echo "========================================"

# Print the last 5 matching lines
if [ $COUNT -gt 0 ]; then
    echo "Last 5 occurrences:"
    grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
fi
```

Concepts Used

- Positional parameters ($1, $2) — Reading arguments passed to the script on the command line
- Default values with ${var:-default} — Providing a fallback if an argument isn't supplied
- File test operators ([-f], [-s], [-z]) — Checking file existence, non-empty size, and empty strings
- while true loop with break — A retry mechanism with an escape condition
- while IFS= read -r LINE — Reading a file line by line, safely handling special characters
- grep -iq — Case-insensitive searching, discarding output (just checking for a match)
- Arithmetic with $(( )) — Counter increment
- tail -n 5 — Getting only the last 5 results

Real-World Importance

Log analysis is one of the most practically valuable things you can do with a bash script. Production servers generate enormous amounts of log data. Tools like Splunk, ELK Stack, and Datadog are essentially sophisticated versions of this concept — parsing log files to find patterns, count events, and surface relevant entries. Knowing how to do this at the command line level gives you a foundation for understanding those tools and for quickly diagnosing problems when fancy tools aren't available.



3.5 Script 5: Open Source Manifesto Generator

The fifth script is interactive and somewhat different in character from the others. It asks the user three questions about their relationship with open-source software, then assembles the answers into a personalized 'manifesto' paragraph and saves it to a text file using file redirection.

```bash
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Devraj Charan (24BAI10717)
# Note on Aliases: You can create a shortcut for this by adding `alias manifesto='./05_manifesto_generator.sh'` to your ~/.bashrc

echo "======================================"
echo " Generate Your Open Source Manifesto"
echo "======================================"
echo ""

# Ask three interactive questions
read -p "1. Name one open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. Name one thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

# Compose the paragraph
PARAGRAPH="On this day, $DATE, I affirm my belief in the open-source philosophy. My daily workflow relies on community-built tools like $TOOL. To me, software freedom ultimately

# Write the paragraph to the output file
echo "$PARAGRAPH" > "$OUTPUT"

echo ""
echo "======================================"
echo "Manifesto saved successfully to: $OUTPUT"
echo "======================================"
# Display the file contents
cat "$OUTPUT"
```

Concepts Used

- read -p — Prompting the user for input and storing it in a variable
- Output redirection (> and >>) — Creating a new file with > and appending to it line by line with >>
- Date formatting with date '+format' — Generating a custom-formatted date string
- Command substitution in filenames — Using $(whoami) and $(date) to create unique output filenames
- cat — Displaying the contents of the generated file back to the user as a preview

Real-World Importance

File redirection is one of the most fundamental and powerful features of the Unix shell. Almost every automation script eventually needs to write output to a file — whether it's a log, a report, a generated config file, or a data export. This script demonstrates the pattern at its simplest, but the same approach scales to generating complex reports, writing configuration files dynamically, or building data pipelines where the output of one script becomes the input of the next.

```
yashw@Yash-PC MINGW64 ~/Downloads/files
$ /c/Users/yashw/Downloads/files/05_manifesto_generator.sh
==================================================
    Open Source Manifesto Generator
    YASHWANT RAJ | 24BCY10176 | VIT Bhopal
==================================================
Answer three questions to generate your manifesto.

1. Name one open-source tool you use every day: libreoffice
2. In one word, what does 'freedom' mean to you? choice
3. Name one thing you would build and share freely: student directed library

Manifesto saved to: manifesto_yashw_20260324.txt

--- Preview ---
### My Open Source Manifesto ###
Student: YASHWANT RAJ | Reg: 24BCY10176 | VIT Bhopal University
Date: 24 March 2026

I believe in open source because tools like libreoffice make my work easier,
faster, and accessible to people who otherwise couldn't afford them.

To me, true freedom means choice – and that is exactly what open source
tries to protect: the freedom to use, study, modify, and share software.

If I had the resources and skills to contribute, I would build student directed
library
and release it freely, so the next person doesn't have to start from zero.

Standing on the shoulders of giants is how all progress works.
Open source is how we make sure the shoulders stay accessible to everyone.

--- End of Manifesto ---

Press [Enter] to exit...
```

Chapter 4 — Conclusion

This capstone project turned out to be more interesting than I expected going in. Choosing LibreOffice meant I got to dig into a piece of software with a genuinely compelling history — a community revolt against corporate control, a clean fork that became arguably better than the original, and a project that has quietly reshaped how governments and institutions think about document standards.

The most surprising thing I learned was how much the license choice matters. Before this project, I thought of software licenses as mostly legal boilerplate. But understanding the difference between the GPL, the MPL, and the MIT license — and why The Document Foundation chose MPL 2.0 specifically — showed me that licensing is actually a strategic decision about what kind of community and ecosystem you want to build around your software.

Working on the shell scripts helped me understand Linux in a way that just reading about it doesn't. Actually writing a loop that iterates through /etc and /var/log and checks permissions, or building a log analyzer that scans real system files, made those directories feel real and meaningful rather than abstract. I'd used the terminal before, but this project made me significantly more comfortable with bash.

The comparison between LibreOffice and Microsoft Office also helped me develop a more nuanced view. I came in thinking the answer was obvious (LibreOffice is free, therefore better), but the honest analysis is more complicated. Microsoft Office 365 is genuinely excellent software for collaborative enterprise work. LibreOffice wins on freedom, cost, Linux support, and format standards. The right answer depends on what you're optimizing for.

If I had to summarize the key insight from this whole project in one sentence, it would be this: open source is not just about saving money — it's about who gets to make the decisions. Oracle could have killed OpenOffice.org. The GPL means no one can kill Git by acquiring its creator. The ODF standard means documents saved today will be readable by software that doesn't exist yet. These are not small things.